

Marum

Մարում

A loan ledger that knows what it does not know.

Date	26 August 2026
Runtime	One always-on Go binary + managed PostgreSQL
Edge	Cloudflare — DNS, TLS, WAF. No logic, no secrets.
Delivery	At-least-once, measured
Financial events	Exactly-once by idempotency key
Market	Armenia · AMD · Armenian and English
Access	60-day trial → 7-day grace → paused. Export and erasure always free.
Telemetry	OpenTelemetry → Grafana Cloud Free tier, permanently . One trace per user action across three processes.
Detail	<code>docs/Marum-Reliable-MVP-Design.md</code> v0.3.1 — full DDL and rationale

1 • What this is

Marum keeps a borrower's loans in one place, reminds them before each due date, records what they paid, and computes a repayment plan. It moves no money, holds no bank credentials, recommends no lender, and contains no AI. Every number is deterministic and traceable to an input.

Four boundaries the design is built on

1. A Telegram update becomes a durable command **before** Telegram receives `200 OK`.
2. User-reported facts are append-only. Balances and payment splits are rebuildable caches.
3. A bank-confirmed snapshot is the authoritative anchor. Marum never invents historical bank state.
4. Message delivery is at-least-once. Rare duplicates are accepted, worded for, and measured.

Out of the MVP

Fee-aware `min_cost` optimisation · variable rates · declining-principal loans · revolving cards · delinquent or restructured loans · cross-currency planning · effective-rate comparison · OCR · bank integrations · Telegram Stars. Each returns only after the ledger and engine are proven against real Armenian schedules.

2 · Architecture

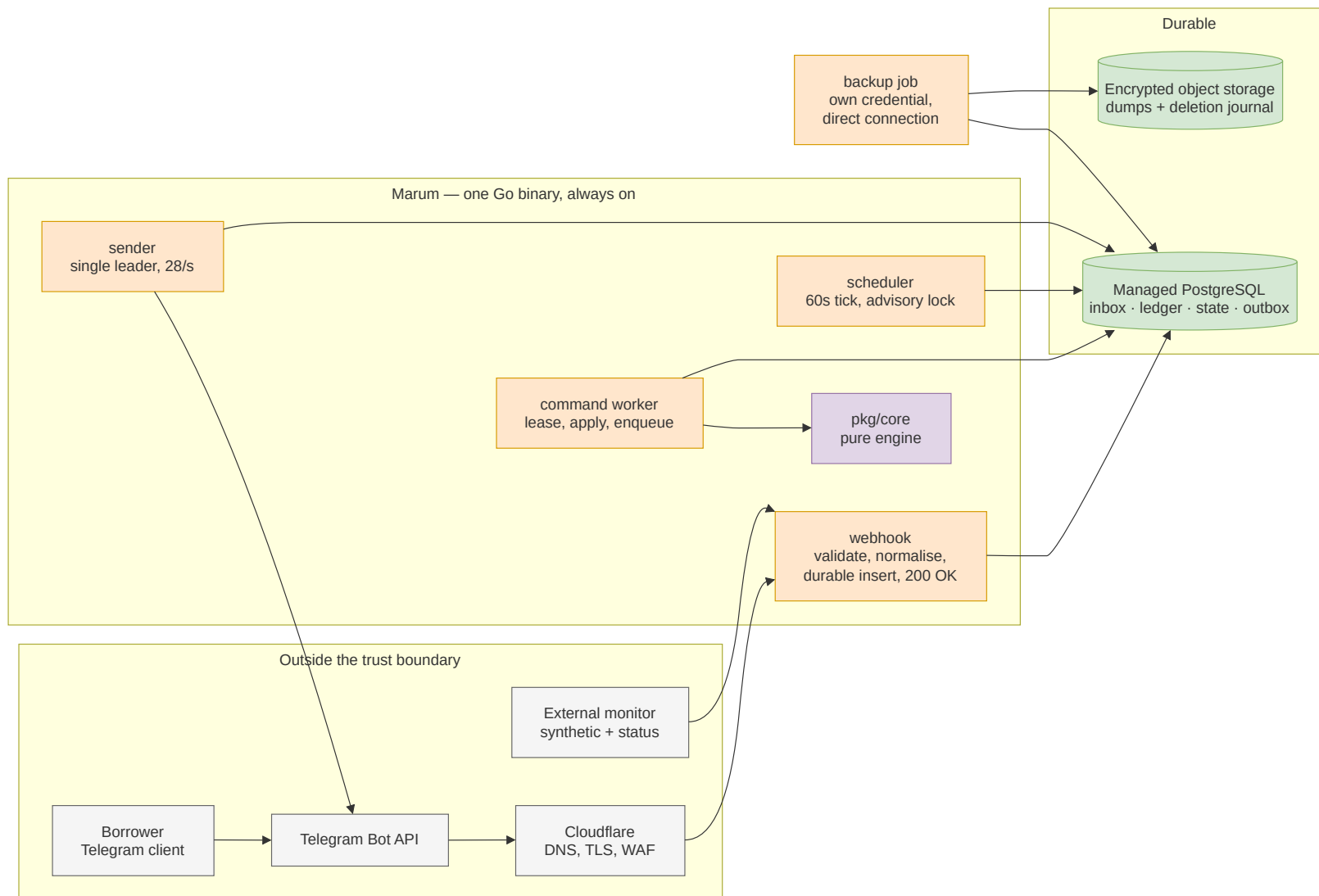


Figure 1 — Deployment. One binary, one database, one language. The backup job holds its own credential and connects directly. The external monitor calls an authenticated status endpoint; it never touches PostgreSQL.

Component	Responsibility
Cloudflare	DNS, TLS, WAF, DDoS. No business logic, no application secrets.
webhook	Validates the secret-token header before reading the body, bounds and normalises the update, inserts a durable command, answers 200 only after commit.
command worker	Leases a command with a fencing token, applies the whole business effect in one transaction, completes the command with the same token.
scheduler	60-second tick under a transaction-scoped advisory lock. Generates reminder occurrences, groups them into deliveries, runs reconciliation. Catch-up tick on startup — it never assumes the previous minute ran.
sender	Exactly one active leader, holding a session-level advisory lock on a dedicated connection. 28 msg/s global, 1 msg/s per chat — meaningful because the bucket is genuinely global.
<code>pkg/core</code>	Pure engine. No database, network, filesystem, clock, randomness, or logging. Time enters as a parameter.
backup job	Separate schedule and credential, direct connection. Dump, encrypt, upload, verify, restore-drill.
PostgreSQL	System of record: durable inbox, ledger, snapshots, derived state, occurrences, delivery outbox.

Two instances may run for webhook availability. Command processing is safely concurrent through leases; sending is not, and the advisory lock enforces that. During a leader handover there is a brief window where both instances believe they hold the role: the send rate may briefly exceed 28/s, which Telegram absorbs as throttling, and a delivery may be attempted twice — the fencing token turns that into one successful mark and one rejection.

No Redis, no broker, no serverless container, no microservices.

3 · Reliability invariants

Implementation requirements, not aspirations. Each is testable, and each has a named failpoint test in

`Marum-Reliable-MVP-Design.md` §17.4.

#	Invariant	Consequence
R1	Acknowledgement means durable acceptance	200 OK only after the command commits. A crash after the response cannot lose the action.
R2	One update causes at most one business effect	<code>telegram_update_id</code> is permanently unique — it survives account erasure with the payload cleared. Every financial event also carries its own idempotency key.
R3	Facts are append-only	Payments, snapshots, voids and billing events are never edited. A mistake is superseded or voided by another row.
R4	Derived state is disposable	<code>loan_state</code> and allocation results can be deleted and rebuilt from the anchor snapshot plus active events.
R5	A bank snapshot outranks Marum's reconstruction	A confirmed snapshot becomes the new replay anchor. Drift is measured, never hidden, and never corrected twice.
R6	No silent financial assumptions	Every projection carries its assumption ledger: day count, rounding, contract version, snapshot date, allocation policy, prepayment behaviour, missing fields.
R7	Delivery is at-least-once	The gap between Telegram accepting and Marum recording <code>message_id</code> cannot be closed. Reminder text reads correctly if it arrives twice.
R8	No network I/O inside a transaction	Telegram, object storage and telemetry calls happen only after commit.
R9	Money is never floating point	<code>int64</code> minor units, checked arithmetic, explicit rounding policy. See §6.
R10	Business dates are not timestamps	Due dates and value dates are date-only. Time zones apply only when converting a local reminder slot to an instant.
R11	Deletion survives restore	An encrypted deletion journal lives outside the backup chain and is replayed after any restore.
R12	Stale or unsupported state cannot produce a confident plan	The eligibility guard is centralised and cannot be bypassed by a handler. See §7.

4 · Processing model

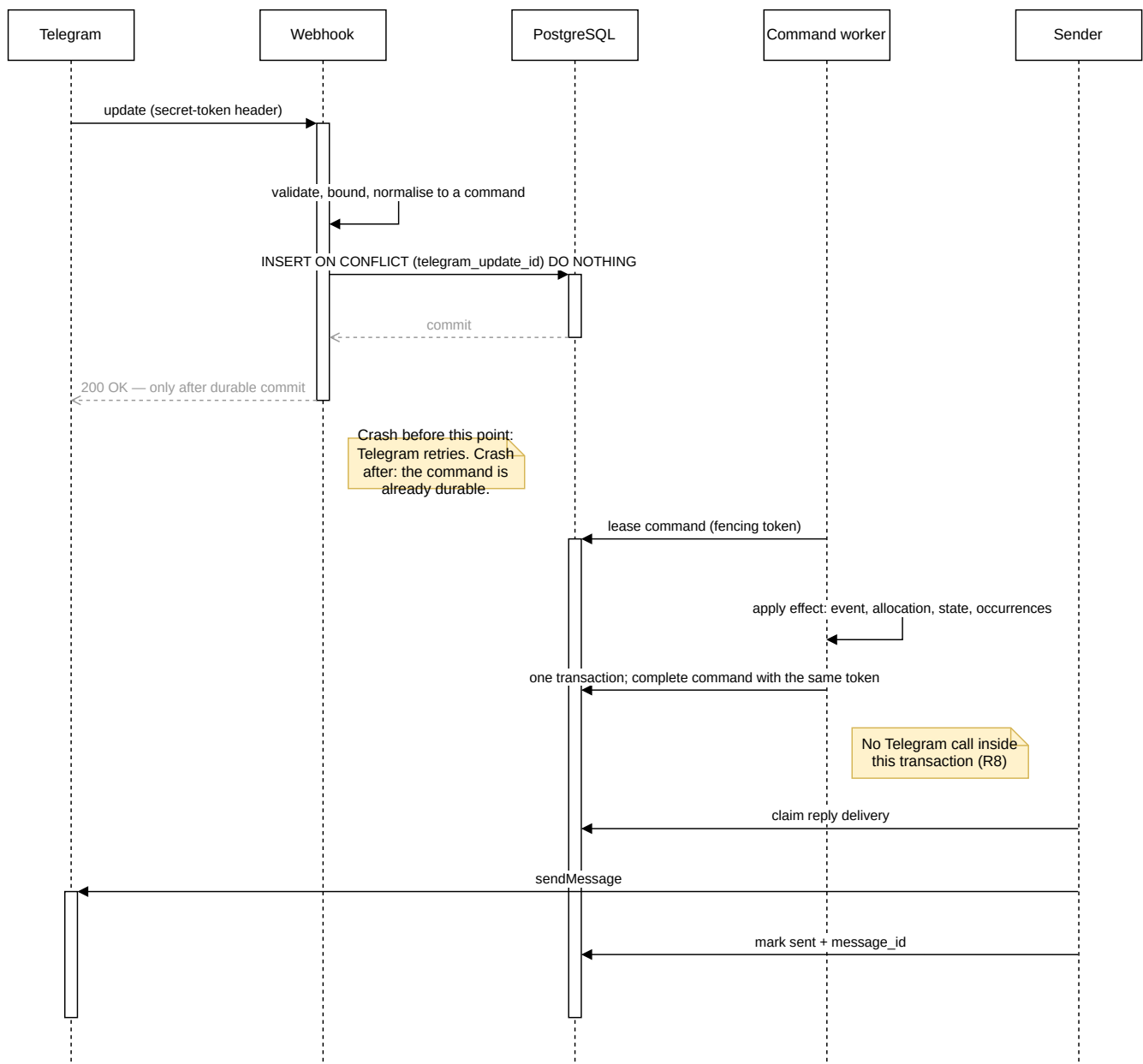


Figure 2 — The durable inbound path. The webhook's only job is to make the update durable. Everything expensive happens afterwards, under a lease, in one transaction, with no network call inside it.

Why a durable inbox rather than a dedup table

A table holding only `update_id` is unsafe: if the row is inserted and the process dies before the action is applied, Telegram's retry is discarded as a duplicate and the user's payment is silently lost. The inbox stores the **normalised command and its processing state**, so a retry finds work still pending.

The business transaction

For a payment-recording command, one transaction does all of this or none of it:

1. Lock the command by its lease token.
2. Lock the loan's event-sequence row.

3. Confirm no financial event already uses this idempotency key.
4. Insert the immutable payment fact.
5. Advance or rebuild derived state.
6. Write a versioned allocation result.
7. Cancel and regenerate affected future reminder occurrences.
8. Enqueue the reply delivery.
9. Complete the command with its fencing token.

The webhook does not store the raw Telegram payload. Input resembling credentials or card data produces a `reject_sensitive_input` command with the value discarded.

5 · Domain model

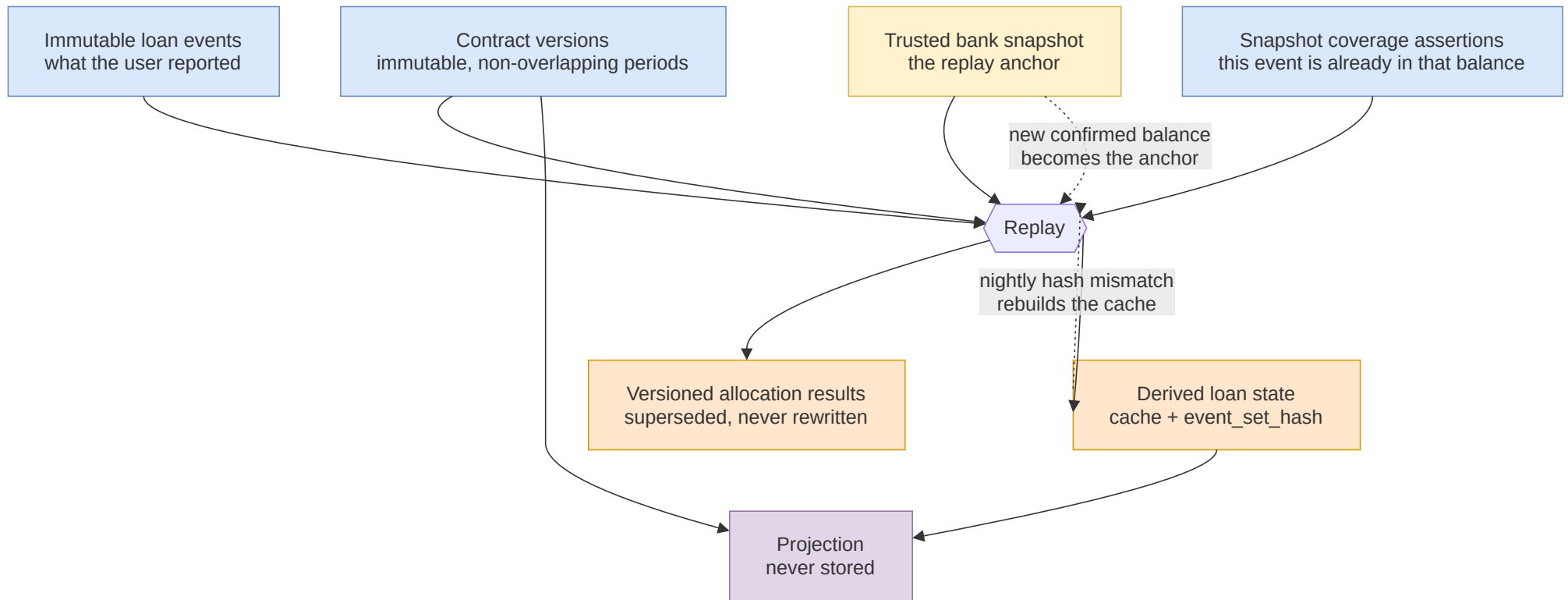


Figure 3 — Facts, anchor, interpretation. Contract versions and reported events are immutable. A confirmed snapshot is the replay anchor. Allocations and state are derived and versioned. Projections are never stored.

Layer	Rule
Loan identity	Holds no financial terms and no balance. Just the enduring user-owned record.
Contract versions	A restructuring, rate change or maturity change creates a new version. Effective periods may not overlap. Every event records the version under which it was interpreted.
Bank snapshots	An observation of lender state at the end of a stated business date, split into independently allocatable buckets: principal, accrued interest, unpaid interest, current fees, overdue fees, penalties, overdue principal, advance credit, next instalment. Trust is <code>user_entered</code> , <code>bank_confirmed</code> , or <code>imported_verified</code> . Only <code>bank_confirmed</code> resets drift.
Loan events	What the user reported, not Marum's interpretation. Five kinds: <code>payment_reported</code> , <code>prepayment_reported</code> , <code>bank_fee_reported</code> , <code>entry_voided</code> , <code>loan_closed_reported</code> . There is no generic correction event — a new snapshot corrects state.
Allocation results	The split into interest, fees, penalties, overdue and scheduled principal, extra principal or advance credit. Stored separately from the fact, stamped with policy version, contract version, engine version, replay generation and confidence. A late earlier event supersedes the result; it never rewrites it.
Derived state	<code>anchor snapshot + active events + policy interpretation</code> , carrying an <code>event_set_hash</code> . Nightly reconciliation recomputes it; a mismatch rebuilds the cache and raises an alert.
Projection	Schedules and plans are function results. No table.

5.1 The two rules that stop double-counting

Snapshot boundary. When an event arrives with `value_date ≤ anchor.as_of`, Marum asks whether the bank balance already included it. Yes writes an immutable coverage assertion and the event is kept for audit but not re-applied. No or unknown sets the loan to `needs_reconciliation` and asks for a fresh balance.

Two orders, never conflated. `recorded_seq` is gapless per loan and records the order Marum learned things. Financial replay orders by `value_date`, then the lender's `bank_order` where supplied, then `recorded_seq` as a deterministic tie-break.

6 · Money, dates, accrual

Concern	Rule
Representation	<code>int64</code> minor units with the currency in the type. AMD has 100 minor units to the dram; whole-dram input is multiplied by 100.
Settlement rounding	Default unit is 100 minor units — lenders settle in whole drams. A different unit requires a real schedule to confirm it.
Rates	Scaled integers in the core (parts per billion); <code>numeric(12,9)</code> in PostgreSQL. Never through <code>float64</code> .
Dates	Date-only in the core. Due-date clamping is centralised: the 31st falls to the last day of a shorter month and returns the following month.
Instalment	Integer bisection for the smallest instalment that closes the dated schedule, using the same builder as projection. The closed-form annuity formula is a test cross-check and an initial bound — never authoritative for irregular periods.

ACCUAL MUST NOT BE EVALUATED IN 64-BIT ARITHMETIC

`interest = principal × rate × days / denominator` overflows `int64` at ordinary local loan sizes:

Loan @ 18%	× rate	× 31 days	int64
4,000,000 AMD	7.2×10^{16}	2.2×10^{18}	fits
16,529,340 AMD	—	9.22×10^{18}	last value that fits
30,000,000 AMD	5.4×10^{17}	1.7×10^{19}	overflow
80,000,000 AMD	1.44×10^{18}	4.5×10^{19}	overflow — 66 bits

At a card rate of 26% the ceiling falls to about 11,443,389 AMD. Compute the numerator in a **128-bit intermediate** (`math/bits.Mul64` with a $128 \div 64$ division), then divide and round once. Do not reorder into `(principal × days) / denom × rate` — that dodges the range problem by rounding twice and silently changes the schedule. Overflow is a typed error, never a wrap.

7 • Eligibility

Planning is gated, but the gate is graded. A single confirmed-and-fresh check in front of every calculation would refuse most users most of the time, and a tool that usually refuses is one people stop opening.

Tier	Condition	May show / must not show
confident	Supported contract, <code>bank_confirmed</code> snapshot within the freshness threshold (initially 35 days), no ambiguous pre-snapshot event, no unsupported overdue or penalty state, known allocation and prepayment policy, successful replay	Plans, savings, debt-free dates, strategy comparison.
indicative	Confirmed snapshot older than the threshold; nothing else wrong	Projections labelled with the snapshot date and its age, next payment, reminders. Not savings figures or a debt-free date stated as fact. Carries a one-tap confirm action that promotes the loan back.
blocked	Ambiguous pre-snapshot event, unsupported overdue or penalty state, unknown allocation policy, or failed replay	The ledger, the reminders, and the exact reason. No projection.

The eligibility function returns machine-readable reasons, and those reasons are what the user sees.

7.1 Safe degradation

Condition	Behaviour
Allocation policy unknown	Record the fact, do not claim a balance, ask for the bank-reported figure and use it as the new anchor.
Unpaid interest, penalties, overdue principal	Reminders continue; optimisation is disabled; ask for an official payoff figure.
Contractual instalment omitted	Solve it from the dated schedule, label it calculated, show the assumption.
Prepayment requires a bank request	Show the operational step; do not reduce projected principal until a later snapshot confirms it.
Contract behaviour unsupported	Keep the ledger and reminders; show no savings and no debt-free date.

8 · Data model

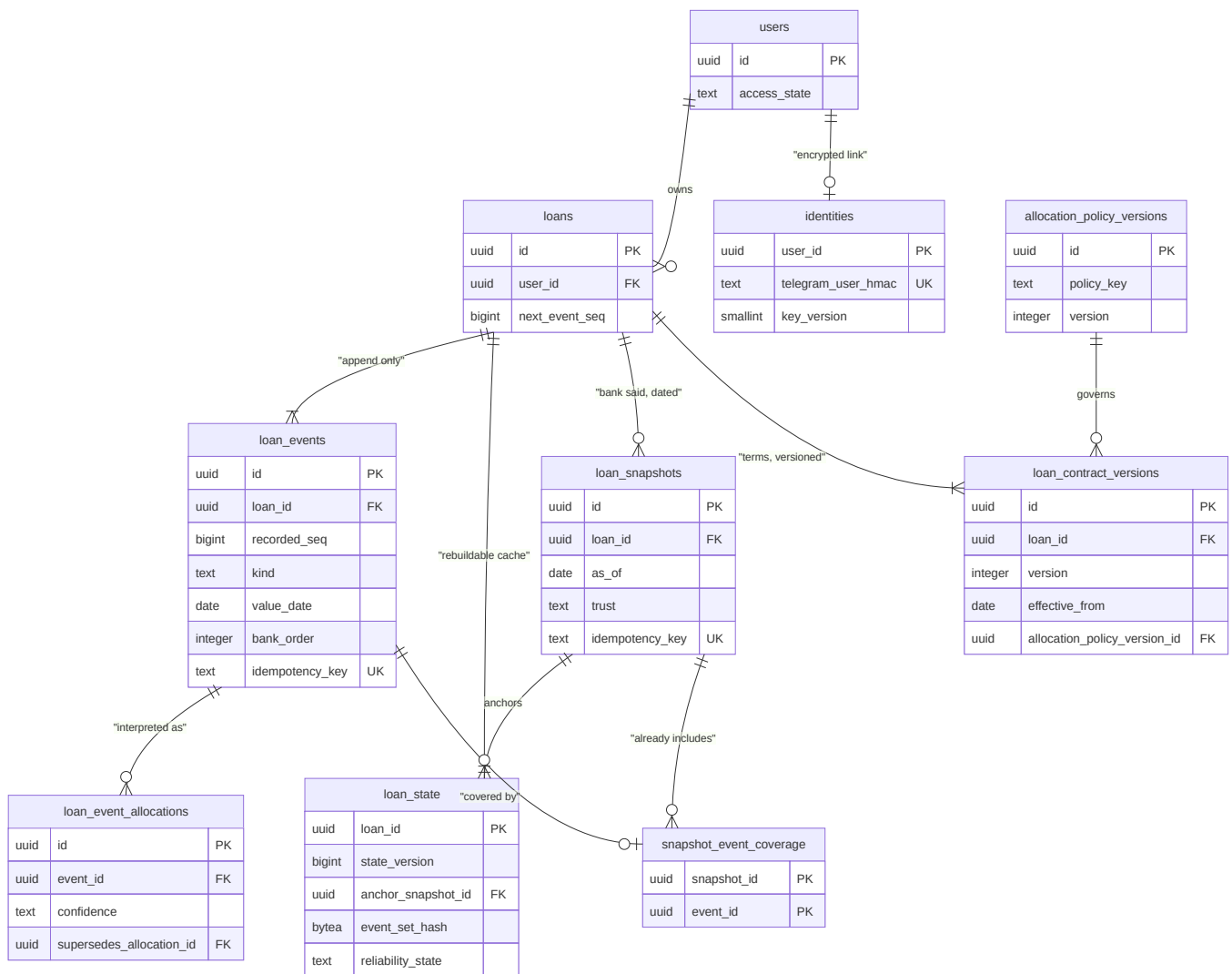


Figure 4 — Core schema. Keys and relationships only; full DDL in `Marum-Reliable-MVP-Design.md` §8. Facts and the anchor on the left, derived interpretation on the right. The application role is denied `UPDATE` and `DELETE` on facts and snapshots; erasure uses a separate narrowly scoped maintenance role.

The remaining tables are operational rather than financial:

Table	Purpose
telegram_commands	The durable inbox. Unique telegram_update_id, status, attempts, next_attempt_at, lease owner and token, plus trace_context so one trace survives the queue (§11.3.1).
conversation_states	Persisted, optimistic-versioned flow position. Advanced in the same transaction that completes its command, so a redelivery cannot advance it twice.
budgets	Monthly amount, reserve floor, month-keyed overrides. The plan itself is not stored.
reminder_rules	Per-loan offsets and local send time.
reminder_occurrences	One row per loan fact due to be reminded. Generated 35 days ahead; cancelled and regenerated transactionally when a payment, snapshot or contract change lands.
notification_deliveries + _items	One Telegram message bundling several occurrences, carrying trace_context for trace continuity. A partial unique index keeps an occurrence attached to at most one live delivery, so a dead delivery releases its occurrences instead of stranding them.
reconciliation_runs	Drift per bucket at each new anchor. Repeated drift on one policy version disables confident planning for it until reviewed.

9 · Reminders and delivery

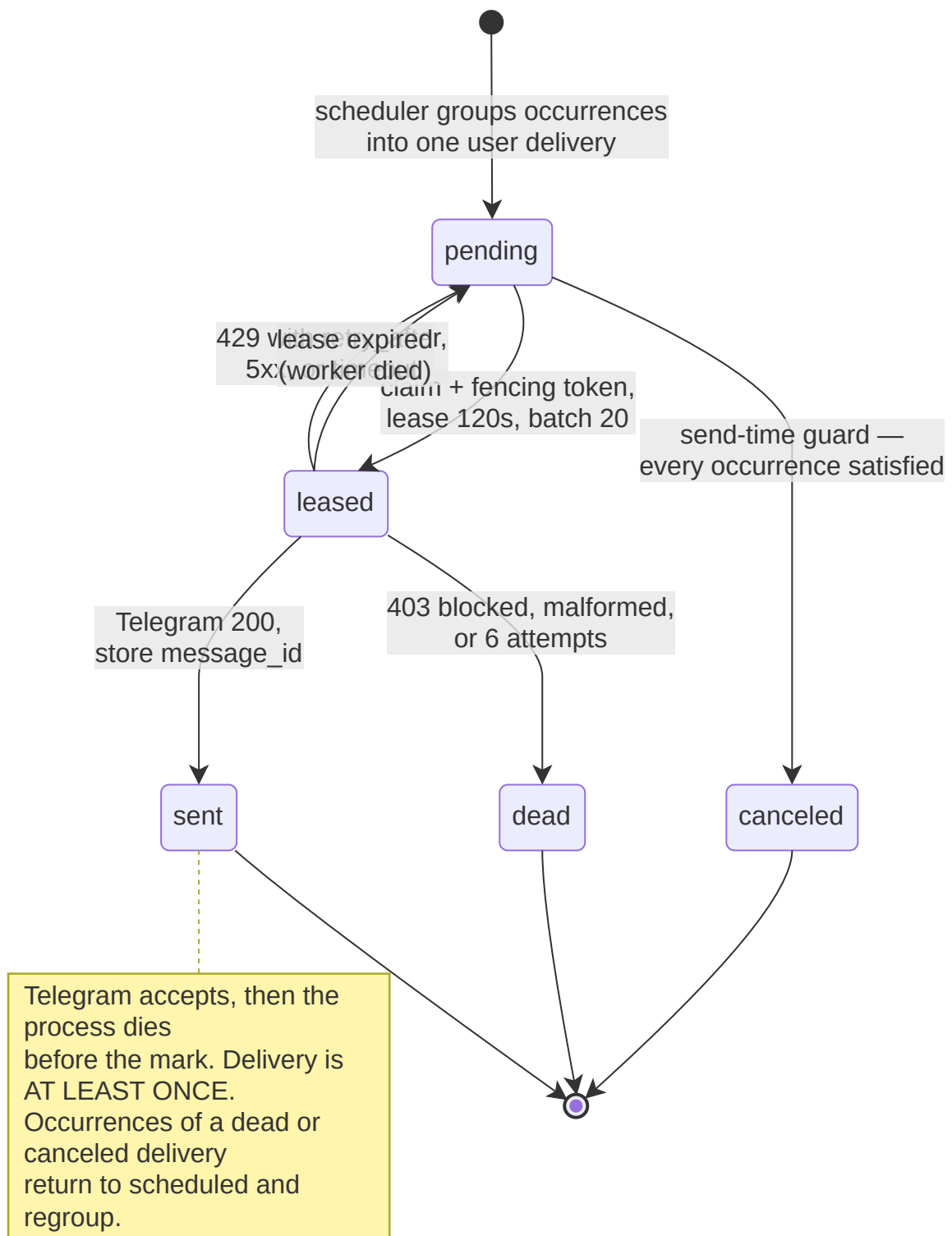


Figure 5 — Delivery lifecycle. Claim under a fencing token, send after commit, mark on success. Everything between the claim and the mark is idempotent, because the process may die at any point in it.

Concern	Rule
Aggregation	The scheduler groups a user's due occurrences into one delivery per local send bucket. The delivery row is created shortly before its bucket, not 35 days ahead, so the frozen payload cannot carry a stale balance.
Quiet hours	21:00–09:00 in the user's timezone. Deferred reminders group into a 09:00 delivery.
Jitter	Deterministic, derived from the occurrence ID — spreads the 09:00 spike without becoming irreproducible.
Send-time guard	Immediately before sending, the sender checks that at least one attached occurrence is still valid; otherwise the delivery is cancelled. A payment can still land inside the remaining window, so reminder text says "if you have already paid, ignore this or record it here".
Retry	429 returns to pending honouring <code>retry_after</code> without blocking other chats. Timeout and 5xx back off exponentially with jitter, six attempts. 403 marks dead and suspends that user's reminders.
Batching	At most 20 deliveries per claim with a 120-second lease, renewed when processing passes half the lease.

Capacity is sized on bursts, not averages. The binding case is the 09:00 release after quiet hours on a salary day. Collapsing a user's due loans into one message before rows are generated is what keeps that within the SLO — without it the 15-minute target breaks at roughly 20,000 users; with it there is headroom past 50,000.

10 • Operations

10.1 Objectives

SLO	Target
Durable webhook acknowledgement	99.9% under 2 s over 30 days
Command completion	99% within 5 s; 99.9% within 60 s
Reminder delivery	99% within 15 minutes of the scheduled slot
Ledger and cache reconciliation	100% nightly; no unresolved mismatch older than 24 h
Recovery point	15 minutes or better once real financial history exists
Restore time	Under 2 hours, measured monthly

10.2 External watchdog

One check that must work when everything else does not: an external monitor calls an authenticated status endpoint whose body comes from bounded queries — last successful tick, oldest pending command, oldest pending delivery, expired leases, dead counts, reconciliation mismatches. PostgreSQL is never exposed publicly.

Alerts route through a **separate bot token**, so a compromise of the product's token cannot silence the alarm about it. Full telemetry design is §11.

10.3 Backups

Two layers. Managed PITR for operational loss — this forces a paid database tier from Phase 2, roughly \$19–25/month, and should be budgeted then rather than discovered later. Plus a nightly `pg_dump -Fc`, separately encrypted, 30 daily and 6 monthly, for logical corruption and provider escape.

A backup counts as successful only after a restore passes: migration verification, constraint checks, application startup, identity decryption for every active key version, `replay = loan_state`, claim queries, deletion-journal application, and a representative read-only journey. Row counts prove nothing.

10.4 Erasure and restore

Before primary deletion the erasure worker writes a subject tombstone to an encrypted append-only journal held outside the backup chain. Any restore proceeds: restore into an isolated database → apply every journal entry included in or newer than the backup → verify erased subject HMACs are absent → run integrity tests → only then promote. Restoring a backup cannot resurrect an erased account.

11 • Observability

Five signals — traces, metrics, logs, continuous profiles and synthetic checks — all pushed to Grafana Cloud and joined by one trace ID. Deep enough to answer "why was this reminder late" and "which function burned the CPU while it happened", and small enough to stay permanently inside the free tier.

11.1 Design rules

#	Rule	Reason
O1	One trace spans the whole user action	A payment crosses the webhook, the command worker and the sender, with durable queues between them. A trace that stops at the webhook explains nothing.
O2	Every log line carries <code>trace_id</code> and <code>span_id</code>	Log → trace and trace → log are one click. Correlation is not a convention people remember.
O3	Histograms carry exemplars	A latency spike on a dashboard links directly to the trace that caused it.
O4	No amount, chat ID, lender name, or raw payload in any signal	Telemetry leaves the trust boundary and lives 14 days in a third-party system.
O5	No unbounded label	Free-tier metrics are billed by active series. A label whose cardinality grows with users is a bill and an outage.
O6	Liveness signals are derived from the database, not from the process	A gauge in a process that has stopped ticking reports nothing. The thing that is broken cannot be the thing that reports it.
O7	Business correctness is instrumented, not only latency	Reconciliation drift and reliability-tier distribution are the metrics that say whether the product is telling the truth.

11.2 Coverage matrix

Every layer of the stack is instrumented, and every layer names what instruments it. A layer with no owner is a blind spot waiting for an incident.

Layer	Instrumented by	What it answers
Telegram edge	Synthetic check + <code>marum_updates_total</code>	Is the public path up when nobody is using it?
HTTP server	<code>otelhttp</code> middleware	Route-level RED, status codes, durable-ack latency
HTTP client (Telegram)	<code>otelhttp</code> transport	Per-call latency, 429 and 5xx rates, retry-after honoured
Command inbox	Hand-written consumer spans + inbox metrics	Lease behaviour, backlog, dead commands
Financial engine	Hand-written internal spans	Replay cost, solver convergence, engine errors
Database	<code>otelpgx</code> tracer + pool metrics	Per-named-query latency, pool saturation, rows touched
Go runtime	<code>contrib/instrumentation/runtime</code>	Goroutines, heap, GC pauses, scheduler latency
CPU and memory attribution	<code>pyroscope-go</code> continuous profiles	Which function, which line, during which span
Scheduler and sender	Hand-written spans + leadership gauge	Tick liveness, exactly-one-sender, delivery lag
Business correctness	Hand-written metrics	Drift, reconciliation mismatches, eligibility mix

AUTO FIRST, HAND-WRITTEN WHERE IT MATTERS

Framework instrumentation (`otelhttp`, `otelpgx`, `runtime`) is free coverage and goes on everywhere. Hand-written spans are reserved for places where the domain vocabulary matters — `core.replay`, `command.process`, `delivery.send` — because "POST /tg/update took 400 ms" is not a useful sentence and "replaying 218 events took 400 ms" is.

11.3 Pipeline

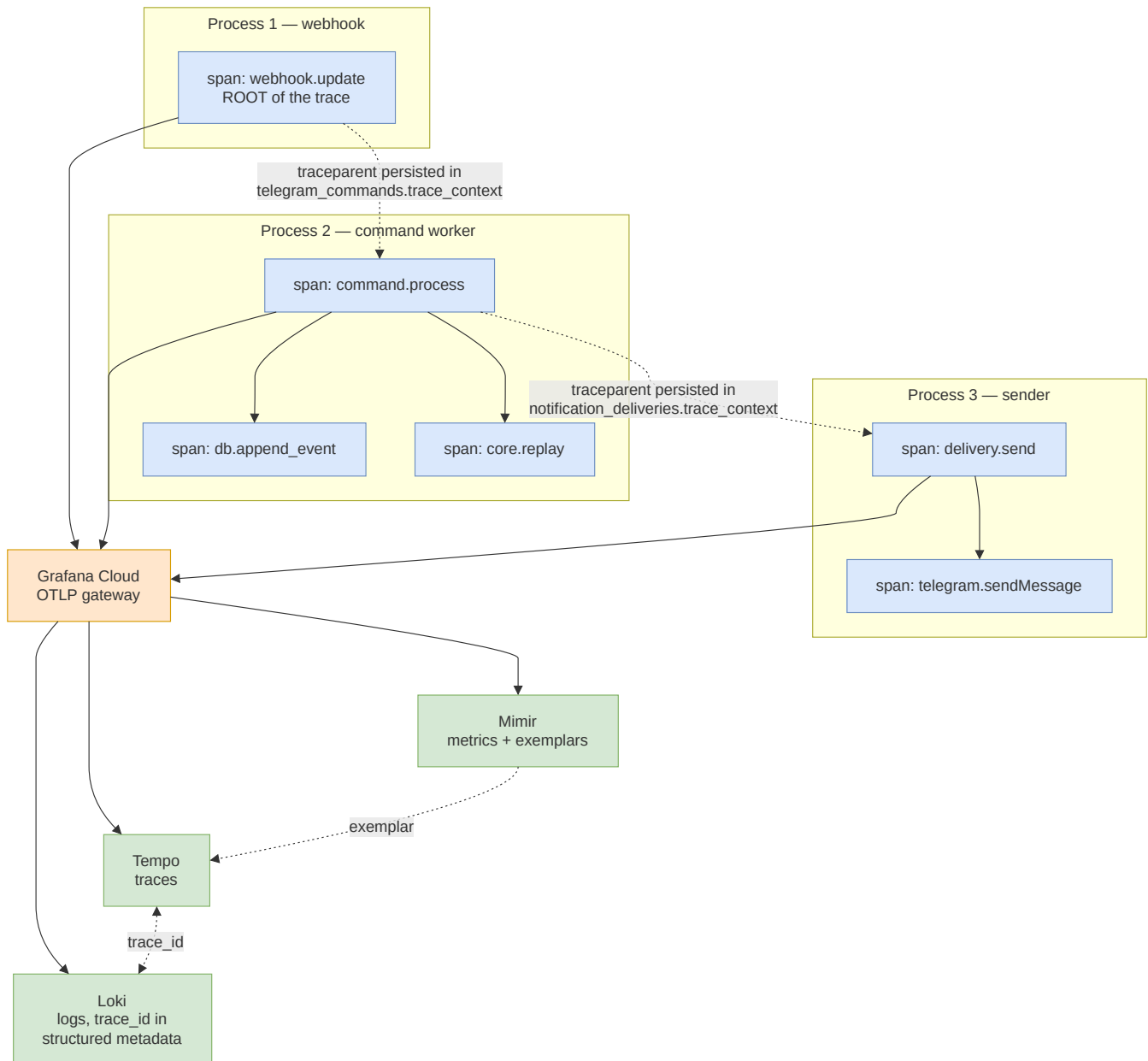


Figure 6 — One trace across three processes. The W3C `traceparent` is persisted alongside the work itself, so the trace survives the durable inbox and the delivery outbox. All three signals leave through a single OTLP gateway; Tempo, Loki and Mimir are joined by `trace_id` and by exemplars.

Traces, metrics and logs travel as OpenTelemetry over OTLP/HTTP to one Grafana Cloud gateway. Profiles use the Pyroscope push protocol to the same stack — OpenTelemetry's profiling signal is not yet stable, and a working profiler today beats a standard one later. No scrape endpoint, no agent, no sidecar: a scrape target would be a second thing to keep alive.

```
# OTLP – traces, metrics, logs
OTEL_EXPORTER_OTLP_ENDPOINT=https://otlp-gateway-<zone>.grafana.net/otlp
OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf
OTEL_EXPORTER_OTLP_HEADERS=Authorization=Basic ${GRAFANA_OTLP_TOKEN} # base64(instanceID:token)
OTEL_SERVICE_NAME=marum
OTEL_RESOURCE_ATTRIBUTES=deployment.environment=prod,service.namespace=marum

# Pyroscope – continuous profiles
PYROSCOPE_SERVER_ADDRESS=https://profiles-<zone>.grafana.net
PYROSCOPE_BASIC_AUTH_USER=<stack-user>
PYROSCOPE_BASIC_AUTH_PASSWORD=<api-token>
```

```
func Init(ctx context.Context, cfg Config) (func(context.Context) error, error) {
    res, err := resource.Merge(resource.Default(), resource.NewWithAttributes(
        semconv.SchemaURL,
        semconv.ServiceName("marum"),
        semconv.DeploymentEnvironmentName(cfg.Env),
        semconv.ServiceInstanceID(cfg.InstanceID),
        // service.version is deliberately NOT here – see 11.6.3.
    ))
    if err != nil { return nil, err }

    tp := trace.NewTracerProvider(
        trace.WithResource(res),
        trace.WithSampler(trace.ParentBased(trace.AlwaysSample())), // see 11.4.4
        trace.WithBatcher(otlptracehttp.NewUnstarted()),
    )
    mp := metric.NewMeterProvider(
        metric.WithResource(res),
        metric.WithReader(metric.NewPeriodicReader(
            otlpmetrichttp.NewUnstarted(), metric.WithInterval(30*time.Second))),
    )
    lp := log.NewLoggerProvider(
        log.WithResource(res),
        log.WithProcessor(log.NewBatchProcessor(otlploghttp.NewUnstarted())),
    )

    // Span profiles: every span carries pprof labels, so Tempo can show the
    // flame graph for one span rather than a process-wide average (11.7.1).
    otel.SetTracerProvider(otelpyroscope.NewTracerProvider(tp))
    otel.SetMeterProvider(mp)
    otel.SetTextMapPropagator(propagation.TraceContext{}) // W3C only, no B3

    // Free coverage: Go runtime metrics – GC, heap, goroutines, scheduler.
    if err := runtimeMetrics.Start(
        runtimeMetrics.WithMinimumReadMemStatsInterval(10*time.Second)); err != nil {
        return nil, err
    }

    prof, err := startProfiler(cfg) // 11.7
    if err != nil { return nil, err }

    return func(ctx context.Context) error {
        return errors.Join(prof.Stop(), tp.Shutdown(ctx),
            mp.ForceFlush(ctx), mp.Shutdown(ctx), lp.Shutdown(ctx))
    }, nil
}
```

Auto-instrumentation is wired at the boundaries every request crosses:

```
// inbound: route-level RED plus a server span per request
handler := otelhttp.NewHandler(mux, "marum",
    otelhttp.WithFilter(func(r *http.Request) bool { return r.URL.Path != "/healthz" }))

// outbound: every Telegram call becomes a client span inside the delivery span
tgClient := &http.Client{Transport: otelhttp.NewTransport(http.DefaultTransport)}

// database: a span per named query, plus pool metrics
pgCfg.ConnConfig.Tracer = otelpgx.NewTracer(otelpgx.WithTrimSQLInSpanName())
otelpgx.RecordStats(pool)
```

The health endpoint is filtered out of tracing deliberately: a check every 60 seconds would otherwise be the most-traced operation in the system and would tell nobody anything.

11.4 Traces

11.4.1 Continuity across a durable queue

The interesting problem is not instrumenting a request — it is that a user action is **three processes separated by two database tables**. The webhook returns before the work happens, and the sender runs minutes later. Two columns solve it:

```
ALTER TABLE telegram_commands      ADD COLUMN trace_context text; -- W3C traceparent
ALTER TABLE notification_deliveries ADD COLUMN trace_context text;
```

The producer injects the current context at insert time; the consumer extracts it and continues the same trace. One `trace_id` then covers webhook → command → reply, which is exactly the object a support question is about.

```
// producer – inside the same transaction that inserts the row
carrier := propagation.MapCarrier{}
otel.GetTextMapPropagator().Inject(ctx, carrier)
cmd.TraceContext = carrier["traceparent"]

// consumer – the lease already returned cmd
ctx = otel.GetTextMapPropagator().Extract(ctx,
    propagation.MapCarrier{"traceparent": cmd.TraceContext})
ctx, span := tracer.Start(ctx, "command.process",
    trace.WithSpanKind(trace.SpanKindConsumer))
```

The fan-in exception. An aggregated reminder delivery bundles occurrences created by several unrelated actions, so it has no single parent. It starts a **new trace with one span link per occurrence**, which keeps the causal graph honest instead of arbitrarily picking one parent.

11.4.2 Span taxonomy

Span	Kind	Notable attributes
<code>webhook.update</code>	server	<code>marum.update_kind</code> , <code>marum.command_kind</code> , <code>marum.duplicate</code>
<code>command.process</code>	consumer	<code>marum.command_kind</code> , <code>marum.attempt</code> , <code>marum.lease_ms</code>
<code>core.replay</code>	internal	<code>marum.events_replayed</code> , <code>marum.anchor_age_days</code> , <code>marum.engine_version</code>
<code>core.schedule</code>	internal	<code>marum.periods</code> , <code>marum.solver_iterations</code>
<code>core.plan</code>	internal	<code>marum.strategy</code> , <code>marum.loan_count</code> , <code>marum.eligibility_tier</code>
<code>db.<query_name></code>	client	<code>db.system</code> , <code>db.operation</code> , <code>marum.rows</code> — never the SQL arguments
<code>scheduler.tick</code>	internal	<code>marum.occurrences_generated</code> , <code>marum.deliveries_grouped</code>
<code>delivery.send</code>	producer	<code>marum.delivery_kind</code> , <code>marum.occurrence_count</code> , <code>marum.attempt</code>
<code>telegram.sendMessage</code>	client	<code>http.response.status_code</code> , <code>marum.retry_after_s</code>
<code>reconcile.loan</code>	internal	<code>marum.drift_bucket</code> , <code>marum.mismatch</code>

11.4.3 What a span may and may not carry

Allowed

- `marum.correlation_id` — short-lived, quotable by the user
- `marum.loan_ref` — first 8 hex of a keyed HMAC of the loan ID: stable enough to follow one loan, not reversible
- Counts, durations, iteration numbers, bucket names
- Enumerations: command kind, strategy, tier, error code

Never

- Any `money.Amount`, in any form
- Telegram user ID or chat ID
- Internal user UUID
- Lender name, loan name, or free text the user typed
- SQL arguments or request bodies

An error span records the error *class* and the correlation ID. The message shown to the user carries the same correlation ID, which is how a support conversation reaches the right trace without anyone pasting a balance into a chat.

11.4.4 Sampling

At MVP scale, **sample everything**. Grafana Cloud meters logs and traces separately and on three axes — processed, written, and retained as GB-months — each with its own 50 GB free allowance per signal:

At 5,000 users	Written / mo	Retained (14 d)	Free	Binds at
Traces — 74,700 spans/day	2.14 GB	1.00 GB-mo	50 GB	~117,000 users
Logs — 26,440 lines/day	0.33 GB	0.15 GB-mo	50 GB	~758,000 users

100% head sampling uses about 4% of the trace allowance and removes an entire class of "the interesting request wasn't sampled" investigation. Revisit at 25 GB written per month, and then move to tail sampling that keeps every error, every span over 1 s, and 10% of the rest — never a blanket ratio.

11.5 Logs

11.5.1 Shape

Structured `slog`, JSON, emitted to stdout for local development and to OTLP in production through one handler chain: `redact → enrich → fan-out`. Enrichment pulls `trace_id`, `span_id` and `correlation_id` from the context, so no call site has to remember them.

```
{
  "time":      "2026-08-26T09:04:12.318Z",
  "level":     "INFO",
  "msg":       "command completed",
  "trace_id":  "4bf92f3577b34da6a3ce929d0e0e4736",
  "span_id":   "00f067aa0ba902b7",
  "correlation": "cr_8f21ba",
  "command":   "record_payment",
  "attempt":   1,
  "duration_ms": 47,
  "loan_ref":  "9c1af0b2",
  "outcome":   "completed"
}
```

Level	Used for	Alerting
DEBUG	Local development only. Disabled in production.	—
INFO	Business transitions: command completed, snapshot confirmed, delivery sent, tick finished.	—
WARN	Recoverable: 429 backoff, lease expiry, replay rebuild, drift within tolerance.	Rate-based only
ERROR	Something is dead or wrong: command dead, delivery dead, reconciliation mismatch, engine error.	Yes

11.5.2 Redaction is a type rule, not a habit

```
// The handler drops, before anything is serialised:
//   - any attribute whose value is a money.Amount (type switch, not name match)
//   - any key in the denylist: amount, balance, principal, payment, chat,
//     telegram_id, token, secret, payload, body
//   - any value longer than 512 bytes
// A unit test asserts a log line built from a fully populated LoanState
// contains none of the digits of its balance.
```

11.5.3 Loki label policy

Loki bills and indexes by label. Only `service_name`, `deployment_environment` and `level` become labels — nine streams in total. `trace_id`, `correlation`, `command` and `loan_ref` go into **structured metadata**, which is queryable and filterable without creating a stream per value. Promoting `trace_id` to a label would create one stream per request and is the single fastest way to break a Loki tenant.

11.6 Metrics

11.6.1 Catalogue

Metric	Type	Labels	Answers
Webhook — is the front door healthy?			
<code>marum_updates_total</code>	counter	kind, result	Traffic, duplicate rate, rejection rate
<code>marum_webhook_duration_seconds</code>	histogram	kind	The 2 s durable-ack SLO
Inbox — is work getting done?			
<code>marum_commands_total</code>	counter	command_kind, outcome	Completion vs retry vs dead
<code>marum_command_duration_seconds</code>	histogram	command_kind	The 5 s / 60 s completion SLO
<code>marum_command_backlog</code>	gauge	—	Are we falling behind?
<code>marum_command_oldest_pending_seconds</code>	gauge	—	Worst-case user wait
<code>marum_command_lease_expired_total</code>	counter	—	Workers dying mid-command
Engine — is the arithmetic sound?			
<code>marum_replay_duration_seconds</code>	histogram	trigger	Replay cost as ledgers grow
<code>marum_replay_events</code>	histogram	—	When to introduce checkpoints
<code>marum_solver_iterations</code>	histogram	—	Bisection converging as expected
<code>marum_engine_errors_total</code>	counter	kind	Negative amortisation, bad contract, unsupported
<code>marum_accrual_overflow_total</code>	counter	—	Must stay at zero. Any increment is a \$6 violation reaching production
Correctness — is Marum telling the truth?			
<code>marum_reconciliation_mismatch_total</code>	counter	reason	Cache disagreeing with the ledger
<code>marum_drift_minor</code>	histogram	bucket	How far our reconstruction sits from the bank
<code>marum_loan_reliability_state</code>	gauge	state	How much of the book is confirmed vs stale
<code>marum_eligibility_tier</code>	gauge	tier	How often users can actually get a plan
<code>marum_snapshot_age_days</code>	histogram	—	Whether the freshness threshold is realistic
<code>marum_allocation_confidence_total</code>	counter	confidence	Estimated vs bank-confirmed splits
Delivery — did the reminder arrive?			
<code>marum_deliveries_total</code>	counter	kind, result	The product's core promise

Metric	Type	Labels	Answers
<code>marum_delivery_lag_seconds</code>	histogram	kind	The 15-minute SLO
<code>marum_delivery_backlog</code>	gauge	—	Burst absorption
<code>marum_telegram_errors_total</code>	counter	code	429 vs 403 vs 5xx need different responses
<code>marum_send_throttled_total</code>	counter	—	Sender at its ceiling
<code>marum_sender_is_leader</code>	gauge	—	Must sum to exactly 1 across instances
Scheduler, database, product			
<code>marum_tick_last_success_timestamp</code>	gauge	—	Dead-man switch
<code>marum_tick_duration_seconds</code>	histogram	—	Tick outgrowing its interval
<code>marum_occurrences_collapsed_total</code>	counter	—	Aggregation actually working (§9)
<code>marum_db_query_duration_seconds</code>	histogram	query	Named sqlc queries only — a bounded set
<code>marum_db_pool</code>	gauge	state	Saturation before it becomes latency
<code>marum_users_by_access_state</code>	gauge	state	Trial → grace → paused funnel
<code>marum_build_info</code>	gauge	version, commit	Which build is running — the only place a version appears

11.6.2 Cardinality budget

Counted as label combinations, histograms at twelve buckets plus `_sum` and `_count`:

Group	Series / instance	Share
Database — 40 named queries × 14	563	49%
Inbox	173	15%
Correctness	84	7%
Go runtime — <code>contrib/runtime</code>	80	7%
Engine	77	7%
Delivery	62	5%
Webhook	54	5%
HTTP server and client — <code>otelhttp</code>	40	3%
Scheduler, product, meta	25	2%
Total per instance	1,158	
Two instances	2,316	23% of the free 10,000

Roughly 7,700 series of headroom, and the number does not grow with users because every label is bounded. Database histograms are half the total and are the first lever if that changes — six buckets instead of twelve frees about 240 series per instance.

11.6.3 The version-label trap

DO NOT PUT SERVICE.VERSION IN RESOURCE ATTRIBUTES

Resource attributes become metric labels. Free-tier billing counts **active series per month**, so a label that changes on every deploy multiplies the whole catalogue by the number of releases:

Deploys / month	Monthly active series	Free tier (10,000)
4	9,264	fits, barely
10	23,160	over — \$6.50 per extra 1k
30	69,480	far over

The version belongs on `marum_build_info` — one series per build — and on spans and **profile tags**, where it is an attribute rather than a series dimension. The same reasoning bans `pod` and `container_id`; `service.instance.id` is stable per deployment slot, not per restart.

11.7 Continuous profiling

Metrics say the p99 moved. Traces say which span. **Profiles say which function, on which line.** Without them every CPU or memory question ends in a guess, or in a local reproduction that does not reproduce.

```
func startProfiler(cfg Config) (*pyroscope.Profiler, error) {
    runtime.SetMutexProfileFraction(0) // off by default – see the type table
    runtime.SetBlockProfileRate(0)

    return pyroscope.Start(pyroscope.Config{
        ApplicationName: "marum",
        ServerAddress:   cfg.PyroscopeAddr,
        BasicAuthUser:   cfg.PyroscopeUser,
        BasicAuthPassword: cfg.PyroscopeToken,
        UploadRate:      60 * time.Second, // see 11.7.2 – not the default
        Tags: map[string]string{
            "env":      cfg.Env,
            "instance": cfg.InstanceID,
            "version":   cfg.Version, // safe here: profile tags are not metric series
        },
        ProfileTypes: []pyroscope.ProfileType{
            pyroscope.ProfileCPU,
            pyroscope.ProfileAllocObjects,
            pyroscope.ProfileAllocSpace,
            pyroscope.ProfileInuseObjects,
            pyroscope.ProfileInuseSpace,
        },
    })
}
```

Profile type	MVP	Reason
CPU	ON	Replay and the bisection solver are the only CPU-heavy paths; a regression shows here first.
Alloc objects / space	ON	Allocation churn in schedule building is the likeliest source of GC pressure.
In-use objects / space	ON	A leak in a long-lived process is invisible until it is an outage.
Goroutines	OFF	Enable while investigating a suspected leak; the runtime goroutine metric already alerts on the symptom.
Mutex, block	OFF	Both add runtime overhead and double profile volume. Temporary settings when contention is suspected — the single-leader sender makes it unlikely.

11.7.1 Span profiles — the part that earns its keep

The profiler tags every sample with the span active at the time, so a slow span in Tempo links to the CPU profile of *that span* rather than a process-wide average. One line of wiring, already shown in §11.3:

```
otel.SetTracerProvider(otelpyroscope.NewTracerProvider(tp))
```

That turns "the plan computation was slow for this user last Tuesday" into a flame graph of exactly that computation, fourteen days later, with no reproduction.

11.7.2 Volume

Profiles are metered like logs and traces — 50 GB written per month on the free tier. Volume is upload interval × profile types × instances, and payload size is not knowable in advance:

Interval	Types	Instances	Pushes / mo	@10 KB	@20 KB	@30 KB
15 s (SDK default)	5	2	1,728,000	17.3 GB	34.6 GB	51.8 GB over
30 s	5	2	864,000	8.6 GB	17.3 GB	25.9 GB
60 s — chosen	5	2	432,000	4.3 GB	8.6 GB	13.0 GB
15 s	10	2	3,456,000	34.6 GB	69.1 GB over	103.7 GB over

60-second uploads, five types, two instances. Safe at any plausible payload size — at most 13 GB of a 50 GB allowance even if profiles are three times larger than expected. The SDK default of 15 seconds is a coin flip against the cap and buys resolution nobody needs for a service handling a few requests per second.

Roll out by measurement, not estimate: enable on one instance, read the real figure in Cost Management after 48 hours, extrapolate, then enable the second. The guardrail alert fires at 40 GB regardless.

11.8 Synthetic monitoring and load

Everything above observes real traffic. Two things observe its absence.

11.8.1 The synthetic check

One HTTP check, two probe locations, 60-second interval, against the authenticated status endpoint. It asserts more than a 200:

Assertion	Catches
HTTP 200 within 2 s	The public path is down, or Cloudflare is not reaching the origin
<code>tick_age_seconds < 300</code>	The scheduler has silently stopped — the failure most likely to go unnoticed
<code>sender_leader = 1</code>	Nothing is sending, or two things are
<code>oldest_pending_command_s < 120</code>	Commands are accepted but not applied
<code>db_reachable = true</code>	The application is up but the database is not

The check runs from outside our infrastructure and depends on none of it. That is the point: it has to work when the thing it watches does not.

11.8.2 Load testing

k6 at phase gates rather than continuously — 500 virtual-user hours per month on the free tier, and four runs of 200 VUs for 15 minutes is about 200 of them. Scenarios are those in §9: the 09:00 quiet-hours release, a salary-day peak, 24-hour outage recovery, a mass 429 wave, and a deploy while commands and deliveries are leased.

11.9 Correlation in practice

Five signals are only worth their cost if moving between them is one click.

Starting point	Path
A latency spike on a dashboard	Histogram exemplar → the exact trace in Tempo → its spans → the flame graph for the slow span → the logs of that span.
An error alert	Alert annotation carries <code>trace_id</code> → trace → the failing span's attributes and events.
A user quotes a correlation ID	Loki filter on structured metadata <code>correlation="cr_8f21ba"</code> → <code>trace_id</code> → the full cross-process trace.
"This loan's balance looks wrong"	<code>loan_ref</code> → Loki → every replay and allocation touching it, with the engine version that produced each.
CPU is high but no single trace is slow	Pyroscope directly, filtered by <code>instance</code> and time window — the case traces cannot answer, and the reason profiles are on.
Memory climbing over days	In-use space profile diffed across the period.

11.10 Dashboards

Dashboard	Panels
Service health	Update rate by result · webhook p50/p95/p99 · command completion vs dead · backlog and oldest pending · database pool and query p99 · error-log rate
Delivery	Reminders sent vs failed · delivery lag percentiles against the 15-minute line · backlog through the 09:00 window · Telegram errors by code · sender leadership (must equal 1) · collapse ratio
Financial correctness	Reconciliation mismatches (target zero) · drift distribution by bucket · reliability-state mix over time · eligibility-tier mix · snapshot-age histogram · <code>accrual_overflow_total</code> (target zero)
Product	Access-state funnel · active loans · second-payment-recorded rate — the retention signal that matters · plan computations by strategy
Runtime	Goroutine count · heap in use and allocation rate · GC pause percentiles · scheduler latency · flame graphs linked from each panel

Dashboards are provisioned as JSON in `deploy/grafana/` and version-controlled. A dashboard built by hand in the UI does not survive the person who built it.

11.11 Alerts

Alert	Condition	Sev	First action
Scheduler stopped	<code>time() - marum_tick_last_success_timestamp > 300</code>	Page	Check instance liveness and the advisory lock; a tick that never runs is silent by nature
No sender leader	<code>sum(marum_sender_is_leader) $\neq$ 1</code> for 2 min	Page	Zero means nothing is sending; more than one means the rate limit is not global
Delivery backlog	<code>marum_delivery_backlog > 200</code> for 15 min	Page	Split by Telegram error code; if 429-dominated, lower the global rate and let it drain
Reminder lag	p99 <code>marum_delivery_lag_seconds > 900</code>	Page	The 15-minute SLO is breaking
Command backlog	<code>marum_command_oldest_pending_seconds > 120</code>	Page	User actions are not being applied
Accrual overflow	<code>increase(marum_accrual_overflow_total[1h]) > 0</code>	Page	Stop showing projections for affected loans. This must never fire
Reconciliation mismatch	<code>increase(marum_reconciliation_mismatch_total[24h]) > 0</code>	Page	The cache disagrees with the ledger; rebuild and find out why
Webhook errors	error ratio > 2% for 10 min	Page	Compare against the deploy annotation; roll back first, diagnose second
Drift rising for one policy	p90 <code>marum_drift_minor</code> above threshold, 7-day window	Ticket	Disable confident planning for that allocation policy version and review it
Series budget	active series > 8,000 (80% of free)	Ticket	Something added a label. Find it before the bill does
Profile volume	> 40 GB written in a rolling month	Ticket	Raise the upload interval or drop a profile type (§11.7.2)
Log or trace volume	> 40 GB written in a rolling month	Ticket	Introduce tail sampling per 11.4.4
Synthetic executions	> 80,000 in a rolling month	Ticket	A probe or a check was added. Only one check with two probes is budgeted (§11.12.2)
Instance count	<code>count(marum_build_info) > 2</code> for 1 h	Ticket	An extra instance is affordable but changes every per-instance meter — confirm it was intended

All alerts carry the runbook link and, where available, an exemplar `trace_id`. They route to a private Telegram channel owned by a **separate bot**: Marum's own alerting must not depend on Marum's bot token.

11.12 Staying inside the free tier

The plan is Grafana Cloud Free, permanently, not a starting point. With all five signals switched on it is still comfortably met — because the risk was never volume.

THE FINDING

Volume is not the constraint. Configuration is. Trace volume binds at roughly 117,000 users and log volume at 758,000. The meters that can be exceeded *next week* are tied to how many things are switched on: synthetic probes, profile upload interval, Grafana seats, and any label somebody adds.

11.12.1 Every metered product, decided

Product	Use	Free allowance	Decision and reason
Metrics (Mimir)	Yes	10k active series, 14 d	2,316 series across two instances, every label bounded.
Logs (Loki)	Yes	50 GB processed / written / retained	0.33 GB written per month. Three labels only.
Traces (Tempo)	Yes	50 GB each	2.14 GB written per month at 100% sampling.
Profiles (Pyroscope)	Yes	50 GB each	4–13 GB at 60-second uploads with five types. Span profiles are the deepest insight available for the price.
Synthetics	Yes	100k API executions	One check, two probes, 60 s — 86% of the allowance. See 11.12.2.
k6	Yes, at gates	500 VUh	~200 VUh per month of burst testing.
Grafana Alerting	Yes	Included	Rules provisioned as code; Telegram contact point on a separate bot.
Visualization seats	Yes	3 active users	One maintainer. A fourth viewer is \$8/month — share snapshots instead.
Application Observability	No	2,232 host hours	The host-hour charge attaches to <i>using the product</i> . Grafana's billing documentation states that building your own dashboards, alerts and workflows incurs no host-hour fee — so we send OTLP, build our own, and this meter stays at zero.
Database Observability	No	2,232 host hours	Same host-hour meter. <code>otelpgx</code> spans plus <code>marum_db_*</code> answer pool saturation and query latency without it.
Frontend Observability	No	50k sessions	No web frontend in the MVP.
Kubernetes Monitoring	No	host + container hours	No Kubernetes.
IRM · Assistant	No	3 users each	Incident response for a one-person team is a Telegram channel and a runbook. Both are seat meters.

11.12.2 The synthetics probe trap

EACH PROBE LOCATION COUNTS AS AN EXECUTION

The free allowance is 100,000 API test executions per month, which is **2.31 executions per minute in total** — across every check multiplied by every probe location. A single check that looks harmless in the UI goes over the moment a third probe is ticked:

Checks	Probes	Interval	Executions / month	Free tier
1	1	60 s	43,200	fits
1	2	60 s	86,400	fits — our configuration
1	3	60 s	129,600	over
2	2	60 s	172,800	over

Configuration: one HTTP check against the authenticated status endpoint, two probe locations (one European, one other, so a single probe outage is not a false page), 60-second interval. 86,400 executions per month, 86% of the allowance, and no room for a second check — which is correct, because a second synthetic would be watching something the first already covers.

11.12.3 Headroom with everything enabled

Meter	Projected	Free	Used	Binds when
Metric active series	2,316	10,000	23%	A 9th instance, or an unbounded label
Traces written	2.14 GB	50 GB	4%	~117,000 users at full sampling
Logs written	0.33 GB	50 GB	<1%	~758,000 users
Profiles written	4–13 GB	50 GB	9–26%	A shorter upload interval or extra profile types
Synthetic executions	86,400	100,000	86%	A third probe or a second check
k6 virtual-user hours	~200	500	40%	Load tests above 200 VUs run monthly
App O11y host hours	0	2,232	0%	Opening the Application Observability UI
Grafana seats	1	3	33%	A fourth person needs a login

Two meters sit near their ceiling and both are configuration, not growth. Everything driven by user count has two to three orders of magnitude of headroom.

11.12.4 The rules that keep it free

- No unbounded metric label, ever.** Reviewed on every pull request; `service.version` stays out of resource attributes (§11.6.3).
- One synthetic check, at most two probes, 60-second interval.** Changing any of the three means redoing the arithmetic.
- Profiles upload every 60 seconds, five types.** Shorter intervals and the mutex, block and goroutine types are temporary investigation settings, reverted afterwards.
- Never open the Application Observability UI.** Using it starts a host-hour meter that sending OTLP alone does not.
- Host-hour products stay off** — Database Observability and Kubernetes Monitoring for the same reason.

6. **Two application instances.** A third is affordable on every meter; a fourth needs the budget re-checked.
7. **Nothing important lives only in Grafana.** Retention is 14 days — incident write-ups, drift investigations and golden fixtures are copied into the repository.
8. **Alert at 80% of every meter** (\$11.11), so the first sign is a ticket rather than an invoice.

11.12.5 What would actually force Pro

In the order they are likely to arrive: a **fourth person** needing a Grafana login (\$8/month), **metric retention beyond 14 days** to compare this quarter's drift against last quarter's, or an **uptime SLA** becoming a commitment to paying customers. None is a volume problem and none arrives before Phase 4.

Pro is \$19/month plus usage. If it becomes necessary the cost is small — but it should be a chosen upgrade, not an overage discovered in a bill, which is the purpose of 11.12.3 and the guardrail alerts.

12 · Release and migration

1. **Expand.** Additive migration, compatible with the running binary.
2. **Verify.** CI runs the *previous* release's integration suite against the expanded schema. This matters more than `up → down → up`.
3. **Deploy** dual-schema code.
4. **Smoke** in production: webhook echo, tick, replay, claim queries, database round trip.
5. **Backfill** in a separate idempotent, resumable, observable job — never inside a migration.
6. **Switch** reads and writes.
7. **Contract** in a later release, once nothing reads the old representation.

Binary rollback never requires a schema rollback. Immutable financial data is never mechanically rewritten without a versioned migration report and a replay comparison. Production and the cloud test environment share a binary, a PostgreSQL major version and a migration path; the test environment exists with its own bot token and database before public beta.

13 • Roadmap

Phase	Scope	Exit gate
0	Collect contracts and official schedules; document lender policies	Ten schedules, four lenders, supported behaviours confirmed to the dram
1	Money and date engine, dated solver, snapshots, immutable events, replay, allocation versions	Golden and property tests pass; late-event and reconciliation cases proven
2	Durable inbox, conversation state, one loan, reminders, payment recording, export and erasure	Ten beta users, two cycles, no lost commands, every discrepancy resolved
3	Multiple loans; highest-rate-first, smallest-balance-first, selected-loan, custom	Real portfolio comparisons reviewed; 10× burst test passes
4	Test environment, Telegram Stars event ledger, entitlement derivation, trial lifecycle	Purchase, renewal, refund, cancellation, chargeback, duplicate and out-of-order cases proven
5	Carried-cash fee-aware optimiser, further repayment types, effective rate	Driven by real contracts and repeated demand; brute-force comparison on small portfolios

14 · Open questions

Question	Needed by	Why it blocks
Day count and rounding behaviour per lender and product	Phase 1	Changes every schedule row
Payment allocation order per product	Phase 1	Decides whether Marum may derive state at all
What a balance's <code>as_of</code> date includes — same-day transactions or not	Phase 1	Defines the replay boundary
Whether excess payments reduce principal, advance instalments, or need a request	Phase 2	Decides whether savings may be shown
Freshness threshold for a confirmed balance	Phase 2	Controls when plans become indicative
Legal retention for subscription records after erasure	Phase 4	Changes erasure semantics
Telegram Stars net proceeds and subscription event behaviour	Phase 4	Required for pricing and entitlement design

15 • Acceptance

Not ready for public beta until every line is true.

- Durable inbox survives crash-after-ack
- A repeated update produces one financial event
- Facts are immutable under the application role
- Anchor plus active events rebuilds state exactly
- Late pre-snapshot events cannot double-count
- Allocations supersede without rewriting history
- Every event ties to a contract version
- Stale or unsupported state cannot produce a confident plan
- Accrual is exact for an 80,000,000 AMD loan over 31 days
- Ten real schedules from four lenders match to the dram
- Occurrences aggregate into one delivery where appropriate
- A dead delivery releases its occurrences
- Command and delivery leases use fencing tokens
- Sender leadership stops on lock loss
- The duplicate-message failpoint is tested and accepted
- 10× burst and outage recovery meet the reminder SLO
- PITR and logical restore drills pass
- Restore does not resurrect erased accounts
- Erasure preserves `telegram_update_id` uniqueness
- Export and erasure work without payment
- One trace covers webhook → command → reply
- No log line contains an amount or a chat ID
- Active series stay under 8,000
- Grafana Cloud usage stays inside every free-tier meter

Marum — MVP system and architecture design v1.0, 26 August 2026. Detailed schema, rationale and revision history in `docs/Marum-Reliable-MVP-Design.md` v0.3.1; coding conventions in `docs/engineering-guide.md`. Marum provides calculations and organisational tools. It does not lend, recommend lenders, refinance, access accounts, move money, or guarantee savings. The lender's agreement and official schedule always take precedence.